

Durable Goals: An Immutable, Evidence-Gated Protocol for Goal Continuity in Long-Running AI-Agent Workflows

Tim Inzitari

Independent Researcher

tim-inzitari · GitHub

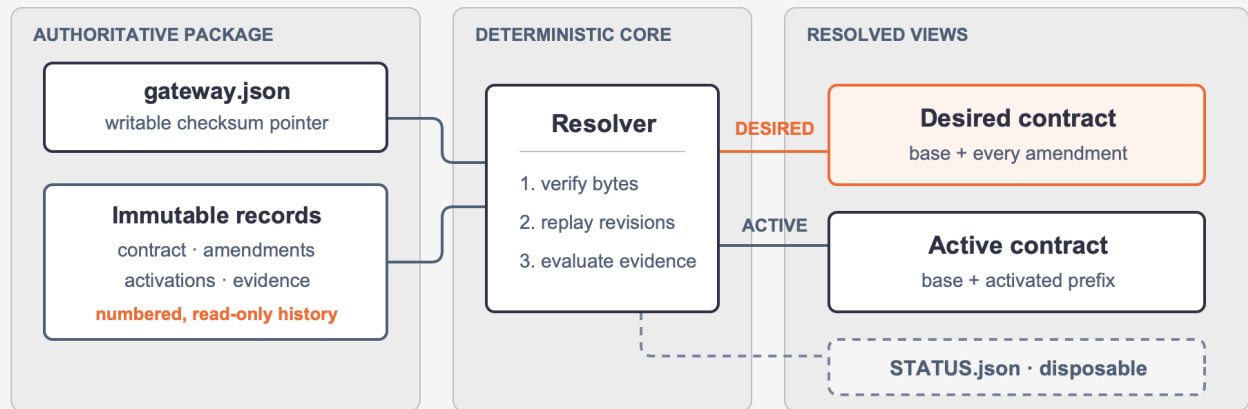
Abstract—Long-running AI-agent work is vulnerable to lost conversational context, silent changes in owner intent, unverifiable completion claims, and duplicated effort across concurrent sessions. This paper presents Durable Goals, a repository-native protocol and dependency-free Python reference implementation for representing authoritative goal state outside any one conversation or process. Version 0.1.1 makes canonical goal records immutable after creation: the initial contract, ledgers, evidence index, and GOAL.md are read-only; each mutation publishes a new numbered, content-addressed history object; and only gateway.json advances to reference the new record set. The resolver verifies and parses the same captured byte snapshot, eliminating a check-then-read substitution window. It deterministically derives a desired contract containing every recorded owner amendment and an active contract containing only the activated amendment prefix. Generated status is disposable and cannot overwrite an authority file. An optional directed acyclic graph exposes ready prompts and stores temporary claims independently per workflow while leaving execution and model choice to an external harness. A current 38-test artifact evaluation exercises these properties. The design assumes a trusted repository boundary: hashes and read-only permissions strengthen local integrity and accident resistance, but do not authenticate authors or establish evidence truth.

Index Terms—AI agents, goal management, provenance, durable state, workflow DAGs, evidence-based completion, prompt loops.

IMMUTABLE GOAL RESOLUTION

Only the gateway advances

Every selected revision remains read-only and checksum-bound.



Example: active r1 fails ($0.87 < 0.90$)

desired r2 passes ($0.87 \geq 0.85$)

r2 still pending

A satisfied guard permits activation; it never activates a revision automatically.

Fig. 1. Durable Goals separates recorded owner intent from activated reality. Canonical revisions are immutable and checksum-bound; gateway.json alone advances to a newly published history set. The resolver applies all amendments to obtain the desired contract but only the activated prefix to obtain the active contract. A satisfied guard does not activate an amendment automatically.

I. INTRODUCTION

An AI agent can continue a task only while it can recover what the task means, which constraints still apply, and what facts establish completion. Conversation history is a poor sole authority for work that spans context-window compaction, process restart, handoff to another harness, or several agent threads. A summary may omit an invariant, combine requested changes with active instructions, or report progress that is no longer supported by the underlying artifacts. Agent-memory systems improve recall, but remembered text and authoritative state are different concerns.

Durable Goals externalizes the small portion of state that must remain normative. The repository contains the objective, invariants, completion predicate, owner amendments, explicit activations, and indexed evidence. A short GOAL.md tells a human or agent where the canonical records are and how to interpret disagreement. A deterministic resolver verifies those records and recomputes the current view. Conversation remains useful context, but it is not allowed to override canonical sources.

The design centers on a distinction that is easy to blur in prose: recorded intent is not necessarily activated reality. An owner may lower a quality threshold, add a requirement, or alter an objective while work is in progress. The decision should be recorded immediately, yet the executing agent may need to continue under the prior contract until a safe boundary. Durable Goals therefore resolves both the latest desired contract and the currently active contract. A separate activation ledger determines which contiguous prefix of amendments has entered effect.

The protocol is deliberately smaller than an autonomous-agent platform or workflow engine. It does not run prompts, select models, assign agents, launch workers, place containers, retry jobs, or reconcile infrastructure. The optional workflow layer computes goal eligibility and emits a prompt that an existing harness may execute. This boundary is a technical constraint, not only positioning: keeping execution outside the protocol permits the same goal files to be consumed by local scripts, coding agents, service managers, or cluster schedulers without embedding their lifecycle semantics.

This paper makes four implementation-backed contributions:

- A repository-native authority model in which canonical records become read-only immutable objects and a writable gateway is the sole advancing pointer to numbered history.
- Deterministic resolution semantics that separate all recorded amendments from the activated amendment prefix, including fail-closed revision, precondition, and evidence checks.
- A restricted predicate language for evidence-backed completion and guarded activation, plus snapshot-consistent verification that parses the exact bytes whose digest was checked.
- An optional acyclic prompt loop whose readiness derives from active completion and whose per-workflow local claims coordinate independent agent threads without becoming model assignment or workload scheduling.

The evaluation is an artifact-level behavioral evaluation, not a performance or security proof. It reports a worked model-refresh trace and the properties exercised by the current 38-test suite. The result is a precise description of version 0.1.1 and its limits rather than a claim that a small local protocol solves general distributed orchestration.

II. PROBLEM MODEL AND DESIGN REQUIREMENTS

A. System Model

The protocol assumes an owner, one or more consuming agents or harnesses, and a trusted repository that carries a goal package. The owner supplies or changes intent. A harness reads the package, performs work using any execution mechanism, and records JSON evidence. The resolver verifies internal references, derives active and desired contracts, and evaluates their predicates. When several goals are present, a workflow file relates their packages through a directed acyclic graph (DAG).

Four types of failure motivate the design. First, conversational state may disappear or become compressed. Second, different prose artifacts may disagree about the objective or current revision. Third, an owner decision may be recorded before it is safe to change the contract under which an ongoing operation is evaluated. Fourth, multiple sessions may discover the same ready work and duplicate it. These failures concern authority and coordination; they do not require the protocol itself to execute a workload.

B. Requirements

R1—Durable resumption: A new process with only the repository must be able to locate the authoritative sources and reconstruct current state without access to the previous conversation.

R2—Deterministic interpretation: The same valid record set must produce the same active contract, desired contract, evidence set, completion result, and workflow ordering. The resolver must parse the exact bytes it verified; ambiguous JSON, non-finite numbers, unknown fields, revision gaps, and checksum mismatches must fail rather than be guessed through.

R3—Controlled evolution: Owner decisions must be append-only, ordered, and separately activated. Later intent must not activate through an earlier pending revision. Optional optimistic preconditions must detect when the value being amended has drifted.

R4—Evidence-backed completion: Completion must be derived from declared JSON evidence rather than status prose. The protocol must verify the identity of the evidence bytes before evaluating a predicate.

R5—Portable containment: Goal references must be relative and confined to the package. The protocol records must remain intelligible without a vendor-specific server, database, model, or editor extension.

R6—Bounded concurrent coordination: Independent ready goals should be discoverable in stable order, and concurrent local claimers should not receive the same goal. Coordination must remain temporary metadata rather than permanent agent assignment.

C. Explicit Non-Goals

Durable Goals does not attempt to authenticate writers, determine whether an evidence receipt is factually honest, provide distributed consensus, lease work across untrusted hosts, or guarantee application-level idempotence. It also does not describe command execution or arbitrary contract code. These omissions prevent a goal authority protocol from accreting the lifecycle, placement, retry, and policy semantics of a general scheduler. Kubernetes, systemd, a CI service, or an agent harness may execute the work; Durable Goals supplies the durable contract and the evidence boundary around it.

III. PROTOCOL ARCHITECTURE

A. Records and Authority

The stable human entry point is GOAL.md. It names the objective, canonical records, source precedence, and resumption procedure. The machine-verifiable entry point is gateway.json, which selects one base contract, amendment ledger, activation ledger, and evidence index using relative paths and SHA-256 digests. Table I summarizes the records.

TABLE I. PROTOCOL RECORDS AND AUTHORITY

RECORD	WRITE MODE	ROLE IN RESOLUTION
GOAL.md	Read-only after creation	Read-first gateway and precedence rule
gateway.json	Atomic pointer update	Selects and checksum-binds canonical records
contract.json	Read-only immutable record	Objective, invariants, completion, delegation metadata
amendments.jsonl	Numbered immutable revision	Ordered owner decisions and activation modes
activations.jsonl	Numbered immutable revision	Adopted contiguous amendment prefix
evidence index + receipts	Content-addressed immutable history	Declared factual inputs bound by checksum
STATUS.json	Regenerated projection	Human-readable progress; never authoritative

This separation prevents generated convenience data from becoming a competing source of truth. STATUS.json includes active and desired completion, pending activation revisions, evidence identifiers, and the hashes from which it was derived, but also carries authoritative: false. Materialization refuses paths inside .dgoal, collisions with GOAL.md, gateway.json, or any gateway-selected canonical record, and overwrite of a file that is not already a matching non-authoritative status projection.

The gateway and evidence digests use the SHA-256 function standardized in FIPS 180-4 [13]. They identify bytes and detect change relative to the recorded digest. They do not identify the author or establish that the repository transport is trustworthy. A signed commit or authenticated storage layer is required when writers do not share the assumed trust boundary.

B. Resolution Semantics

Let C0 be a valid base contract at revision 1, and let A = (a2, ..., an) be a contiguous amendment sequence. Each amendment is a constrained sequence of set or remove operations over RFC 6901 JSON Pointers [11]. Let k be the highest activated revision, with $1 \leq k \leq n$. Valid activation records must correspond exactly to the prefix (a2, ..., ak). The resolver computes

$$C_d = \text{fold}(\text{apply}, C_0, [a_2, \dots, a_n]), \quad (1)$$

$$C_a = \text{fold}(\text{apply}, C_0, [a_2, \dots, a_k]). \quad (2)$$

Cd is the desired contract and Ca is the active contract. The resolver applies amendments in order, updates the synthetic revision field after each application, and validates both resulting contracts. If an amendment contains an expect value, the operation proceeds only when the referenced prior value matches exactly. This is optimistic semantic concurrency: it detects that the intended edit target has changed even if the JSON document remains structurally valid.

Amendments cannot replace the document root or mutate the schema, goal_id, or revision identity fields. Array indices reject negative notation and leading zeros; invalid pointer escapes and missing parents fail resolution. Input JSON rejects duplicate object keys so two parsers cannot choose different meanings for the same purported record.

The activation ledger is intentionally separate from the amendment ledger. An amendment records what the owner wants; an activation record states that a specific next revision has entered effect. Prefix validation prevents revision r4 from activating while r3 remains pending. The separation represents delayed adoption directly instead of encoding it in comments or relying on a timestamp heuristic.

C. Predicates and Evidence

Contracts and safe-boundary amendments use a small recursive predicate language. Composition operators are literal, all, any, and not. Evidence leaves name an evidence identifier and a JSON Pointer field, then apply exactly one of equals, gte, lte, or exists. Numeric comparators reject booleans. An undeclared evidence identifier is a validation error; a missing field causes the affected predicate leaf to be unsatisfied and produces a structured explanation.

For a contract C and a checksum-verified evidence map E, completion is

$$\text{complete}(C, E) = \text{eval}(C.\text{completion}, E). \quad (3)$$

The deliberately restricted language has two advantages. First, evaluation is deterministic and easy to reproduce in another implementation. Second, contracts cannot embed arbitrary executable code. The cost is expressiveness: complex temporal or domain policies must be reduced to application-produced evidence fields or handled outside the protocol.

Evidence enters through the evidence-add command. The writer captures the source bytes once, strictly parses that snapshot, stores those same bytes in content-addressed package history, and advances the gateway's evidence-index reference. Resolution likewise reads each referenced file once, verifies the digest of the captured bytes, and parses that same in-memory snapshot. This closes the check-to-use gap in which a path could previously be replaced between verification and parsing. An activation may additionally bind the exact evidence digests used to satisfy its boundary condition; later resolution rejects a stale or unknown binding.

D. Explicit Activation

Activation modes are manual, immediate, and next safe boundary. In version 0.1.1, every amendment still requires an explicit activation operation; immediate means no additional predicate gate, not silent adoption during amendment recording. The activate command accepts only the first pending revision. For a safe-boundary activation, it evaluates the amendment's declared condition against verified evidence and refuses activation when the guard is false. It then publishes a new immutable activation-ledger revision containing the revision, time, and any required or selected evidence digests.

This detail matters in the bundled example. A release receipt already satisfies the revision-2 safe-boundary condition, but revision 2 remains pending because activations.jsonl is empty. A condition describes when activation is permitted; it does not perform activation.

E. Write Publication and Containment

Mutations hold a short advisory filesystem lock scoped to the package. Initialization writes GOAL.md, contract.json, amendments.jsonl, activations.jsonl, and evidence-index.json through the immutable writer and removes all write bits (mode 0444); gateway.json remains writable because it is the advancing reference pointer. Subsequent owner changes never edit a selected canonical record in place. The writer constructs and validates a complete candidate, serializes a new numbered content-addressed object under .dgoal/history, creates it exclusively, makes it read-only, flushes and fsyncs it, and fsyncs the containing directory. Only then does it write a temporary gateway, fsync the file, replace gateway.json with os.replace, and fsync the directory. If an identical history path already exists, its bytes must match and its read-only mode is restored.

Immutability is therefore represented by both protocol structure and a local filesystem guard. The structure is authoritative: old gateway-selected bytes remain addressable by their digest and a mutation advances the pointer to a newly published object. Read-only permissions add protection against accidental editing but are not a security boundary against a process that owns the files. Generated status and temporary workflow claims are deliberately excluded from immutable authority.

Internal write targets are resolved under the package root. Absolute paths, parent traversal, and references that resolve outside the package are rejected. The writer also inspects path components and refuses symbolic-link traversal for its managed internal state. These measures describe the reference implementation's local publication strategy; they are not a general proof of crash consistency across every operating system, network filesystem, or storage controller.

IV. OPTIONAL WORKFLOW PROMPT LOOP

The single-goal protocol is complete without workflow.json. When a project contains several goals, the optional workflow layer defines a DAG $W = (V, E)$, where each node names a validated goal gateway and each edge is a dependency. Validation rejects missing endpoints, duplicate identifiers, self-edges, package escape, goal-identity mismatch, and cycles. A sorted Kahn traversal produces stable topological order.

For node v , the workflow considers only the active completion result of its goal package. Desired completion cannot release downstream work. Ignoring claims for the moment,

$$ready(v) = \neg complete(v) \wedge \forall u \in pred(v): complete(u). \quad (4)$$

A complete node is completed; an incomplete node with all predecessors complete is ready; otherwise it is blocked. workflow next walks stable topological order and emits the first ready node's GOAL.md prompt, or every ready prompt with --all. Repeated calls return the same unfinished goal because the surrounding harness—not the workflow—must perform work and record evidence.

Claims provide bounded local coordination. The claim command acquires the workflow lock, recomputes readiness, and records the first ready node under a caller-supplied identifier. Each workflow stores a separate claim file, keyed by workflow identifier, beneath the package's temporary-state directory. Thus, two workflows in one directory cannot share or overwrite one claim map. Workflow mutation also prunes claims for nodes that no longer exist. A claimed node disappears from the ready set, allowing another concurrent thread to take an independent branch. Claims do not expire implicitly; abandonment requires explicit release by the same claimant.

OPTIONAL PROMPT LOOP

The protocol exposes work; the harness executes it

Readiness uses active completion only.



Fig. 2. The optional DAG derives readiness from active completion and emits an authoritative goal prompt. The external harness owns execution and model choice; resulting receipts re-enter the relevant goal package as evidence.

The protocol therefore uses “claim” in a narrower sense than a scheduler uses “assignment.” It has no worker registry, resource model, queue capacity, retry policy, or runtime heartbeat. Compared with distributed coordination systems such as ZooKeeper [8], its advisory lock and JSON claim file are explicitly single-repository, local coordination mechanisms.

V. REFERENCE IMPLEMENTATION

A. Modules and Schemas

The reference implementation is version 0.1.1, requires Python 3.11 or later, and declares no runtime dependencies. Six JSON Schemas describe gateway, contract, amendment, activation, evidence-index, and workflow records. The executable validator remains hand-written so it can apply semantic rules beyond structural schemas, including revision continuity, activation-prefix ordering, package confinement, evidence declaration, and rejection of NaN and Infinity throughout predicate and amendment values.

The implementation is divided into small modules. `io.py` performs strict JSON/JSONL loading, canonical serialization, local-path resolution, and digest verification. `validate.py` enforces record and predicate rules. `pointers.py` implements amendment operations and expect preconditions. `evidence.py` evaluates predicates and collects referenced evidence identifiers. `resolve.py` constructs active and desired views. `writer.py` records amendments, activations, evidence, status, and package initialization. `workflow.py` validates DAGs, resolves node states, emits prompts, and manages claims. `cli.py` exposes these operations as the `dgoal` command.

The command surface has four groups. Read operations `validate`, `resolve`, `display status`, and `verify evidence`. Goal mutations `initialize` a package, `amend` it, `activate` the next revision, `materialize status`, or `record evidence`. A compact chain helper adds a completion transition between two contracts. Workflow commands `initialize` and `mutate` a DAG, `inspect status`, `emit prompts`, `claim ready nodes`, and `release abandoned claims`.

B. Cross-Harness Entry Convention

The universal integration is intentionally one instruction: before acting on a new or resumed goal, read `GOAL.md` completely and then read the canonical sources it names for the current action. No plugin is required to interpret the files. A repository-scoped update-durable-goal skill is included as an optional interface that translates natural-language owner requests into CLI mutations; it is not part of the protocol’s authority model.

The protocol artifacts—Markdown, JSON, JSONL, relative paths, and SHA-256 references—are language-neutral in representation. At present, however, only the Python resolver is supplied and no cross-language conformance corpus exists. “Language-neutral” therefore describes the record format and intended interoperability boundary, not experimentally demonstrated equivalence among independent implementations.

C. Fail-Closed Behavior

The CLI maps protocol errors to a nonzero exit and concise diagnostic. Resolution stops on malformed digests, tampered records, invalid schemas, duplicate JSON keys, undeclared evidence, revision gaps, non-prefix activation, or path escape. Workflow mutation validates the candidate graph before advancing its revision, so an attempted cyclic edge leaves the prior workflow bytes unchanged. Writer tests similarly assert that a competing non-waiting writer fails without changing the gateway.

Fail-closed behavior protects interpretation, not the truth of application data. A syntactically valid receipt claiming accuracy 0.99 remains a valid input if a trusted writer indexed it; the protocol cannot independently rerun the evaluation. Applications that require stronger provenance must authenticate producers or connect receipts to an external attestation system.

VI. ARTIFACT EVALUATION

A. Method

The reported artifact is the version-0.1.1 working tree based on Git revision 02ff55a. Tests were executed on arm64 macOS 26.2 with Python 3.11.15 using:

```
PYTHONPATH=src python3 -m unittest discover -s tests -v
```


The observed run completed 38 tests in 0.218 s. This time is reported only to make the run identifiable; it is not a benchmark. The repository contains no coverage report, performance suite, fault-injection campaign, distributed-filesystem test, or scalability experiment.

TABLE II. AUTOMATED TEST DISTRIBUTION

TEST MODULE	COUNT	PRIMARY PROPERTIES
Resolver	16	Integrity, verified-byte snapshots, revisions, strict JSON
CLI	7	Output semantics, mutation parsing, prompt scope
Workflow	7	Fan-in, cycles, per-workflow claims, release, confinement
Writer	8	Immutable files, safe status, activation, links, contention
Total	38	All tests passed in the reported run

B. Model-Refresh Trace

The bundled example begins with revision 1: promote a validated model without losing provenance. Completion requires `evaluation.metrics.accuracy ≥ 0.90` and `release.activated = true`. The evidence index binds an evaluation receipt with accuracy 0.87 and a release receipt with `activated = true`.

Revision 2 records an owner decision lowering the desired accuracy threshold from 0.90 to 0.85. Its `expect` precondition names the prior value 0.90, and its next `safe_boundary` condition requires the release receipt to be activated. No activation record is present.

Resolution therefore produces different but noncontradictory results. The active contract remains revision 1 and is incomplete because 0.87 is below 0.90. The desired contract is revision 2 and is complete because 0.87 meets 0.85 and release activation is true. Revision 2 remains pending. This trace demonstrates all three distinct statements: the new intent is recorded, its guard is currently satisfied, and it is not yet active.

C. Behavioral Property Matrix

TABLE III. BEHAVIORAL PROPERTY MATRIX

SCENARIO	EXPECTED / OBSERVED RESULT
Contract or receipt bytes change	Digest mismatch; resolution stops
File changes after digest verification	Previously captured verified bytes are parsed
JSON contains NaN or Infinity	Strict loader and semantic validator reject it
Amendment or activation skips a revision	Noncontiguous or non-prefix history rejected
<code>expect</code> value has drifted	Amendment application refused
Reference escapes package or traverses a managed symlink	Resolver or writer refuses the target
Initialized authority file is inspected	No write bits are present; gateway stays writable
Status path names an authority file	Materialization refuses the collision
New workflow edge creates a cycle	Mutation fails; prior bytes remain
Two threads claim two ready roots	Distinct nodes are returned
Two workflows share a directory	Separate workflow-id claim files are retained

The concurrent-claim test uses exactly two Python threads and two independent ready goals. It asserts that the returned node set is {one, two}, both workflow nodes become claimed by distinct caller identifiers, and no ready prompt remains. This establishes the intended lock-serialized behavior for the exercised local case. It is not evidence of throughput, fairness, multi-host safety, or performance under contention.

D. Interpretation and Threats to Validity

The tests are tightly aligned with stated invariants, and the model-refresh example exercises the central active-versus-desired distinction. Nevertheless, all evidence is produced by the implementation’s own test suite. There is no independent resolver, formal semantics proof, mutation-testing report, or conformance corpus. The artifact evaluation should therefore be read as reproducible behavioral evidence for the current implementation, not a proof that every filesystem interleaving or malformed input is covered.

VII. RELATED WORK AND STANDARDS BASIS

Belief–Desire–Intention architectures treat intentions as an agent’s adopted deliberative state [2]. Durable Goals is not a cognitive architecture, but its active contract has a related practical role: it distinguishes currently adopted instructions from the larger set of recorded owner desires. Its contribution is a repository-level lifecycle for this distinction across sessions, not a new logic of rational agency.

Generative Agents [3] and MemGPT [4] address persistence and retrieval of experience across long interactions. Those systems help an agent remember. Durable Goals instead narrows persistence to normative state: what objective governs, which changes have entered effect, and what indexed evidence satisfies completion. The systems are complementary because recalled context may inform work without becoming authoritative.

W3C PROV-DM provides a general model for entities, activities, agents, and provenance relationships [5]. in-toto uses signed, linked metadata to verify software-supply-chain steps [6]. Durable Goals adopts a much smaller local mechanism: checksum-bound records and receipts plus explicit owner-decision and activation histories. It does not offer PROV’s general interchange model or in-toto’s authenticated supply-chain guarantees.

Workflow languages such as CWL specify steps, data dependencies, and execution semantics across platforms [7]. Durable Goals’ DAG is intentionally less expressive: nodes are independently resolved goal packages, an edge requires predecessor completion, and the output is a prompt rather than a process invocation. ZooKeeper [8] illustrates the substantially stronger machinery required for distributed coordination; the reference implementation’s advisory locks do not attempt that problem.

Agent interoperability protocols occupy another adjacent layer. A2A standardizes discovery, communication, and collaborative task exchange between opaque agent systems [9], while the Model Context Protocol standardizes interactions with tools and resources [10]. Durable Goals does not define transport or peer discovery. It can be carried through such systems, but its function is the durable owner-governed lifecycle of intent, activation, evidence, and completion.

The record syntax also relies on established standards. JSON Pointer supplies unambiguous paths into JSON documents [11], RFC 3339 supplies offset-bearing timestamps [12], and FIPS 180-4 specifies SHA-256 [13]. The activation design is loosely analogous to the separation between long-lived transactional work and controlled boundaries in sagas [14], but Durable Goals neither executes subtransactions nor defines compensation.

VIII. LIMITATIONS AND FUTURE WORK

The strongest limitation is trust. SHA-256 detects that bytes differ from a recorded digest, and read-only mode deters accidental edits, but a privileged or file-owning malicious writer can change permissions and replace both a record and its gateway digest to create a self-consistent false history. Version 0.1.1 therefore assumes the repository and CLI invocation are inside a trusted owner boundary. Signed authority records, authenticated commits, or another trusted transport are needed across unequal writers.

Evidence integrity is not evidence truth. Application-specific schemas, producer identities, reproducible commands, or remote attestations remain outside the protocol. Likewise, the writer’s advisory locks and atomic replacements are local filesystem mechanisms; behavior over network filesystems, object stores, Windows, and partial hardware failure has not been evaluated.

Claims are explicit and do not expire. That avoids silent duplicate work but leaves a stopped claimant able to block a ready goal until a user or harness releases it. Future versions could add an opt-in lease protocol only if they can preserve visible ownership and avoid broadening the core into a scheduler.

The predicate language is auditable but intentionally small. It has no temporal operators, quantifiers, cross-receipt joins, or arbitrary expressions. Complex policies require preprocessing into evidence or a protocol extension. Delegation fields exist, but delegation resolution, supersession, cancellation, and revocation are not yet first-class end-to-end semantics.

Finally, portability has not been validated through independent implementations. A language-neutral conformance corpus should specify successful resolutions and exact failure cases for duplicate keys, pointers, amendments, activations, evidence, DAG ordering, and claim-independent workflow states. Cross-platform stress tests, signed-authority records, and longitudinal studies of real agent handoffs are higher-value next steps than expanding execution features.

IX. CONCLUSION

Durable Goals defines a small boundary between conversation and authority. A readable, read-only GOAL.md leads to immutable checksum-bound canonical records; owner changes publish new numbered history instead of editing prior authority; a separate activation prefix determines what is in effect; verified byte snapshots drive completion; and generated status remains disposable. The optional DAG exposes eligible prompts while leaving execution to existing harnesses.

The central result is not a new scheduler. It is a deterministic, append-only way to say, after a conversation or process disappears, which goal is desired, which immutable revision is active, what exact evidence bytes are being trusted, and which independent goal is eligible next. The version-0.1.1 artifact demonstrates these semantics under its local trusted-repository model and makes the remaining gaps—authentication, evidence truth, distributed coordination, and conformance—explicit.

REFERENCES

- [1] T. Inzitari, “durable-goals,” version 0.1.1, GitHub, 2026. [Online]. Available: <https://github.com/tim-inzitari/durable-goals>
- [2] A. S. Rao and M. P. Georgeff, “BDI agents: From theory to practice,” in Proc. First Int. Conf. Multiagent Systems, 1995, pp. 312–319. [Online]. Available: <https://cdn.aai.org/ICMAS/1995/ICMAS95-042.pdf>
- [3] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in Proc. ACM UIST, 2023, Art. no. 2, pp. 1–22, doi: 10.1145/3586183.3606763.
- [4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023. [Online]. Available: <https://arxiv.org/abs/2310.08560>
- [5] L. Moreau and P. Missier, Eds., “PROV-DM: The PROV data model,” W3C Recommendation, Apr. 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>
- [6] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in Proc. 28th USENIX Security Symp., 2019, pp. 1393–1410. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>
- [7] Common Workflow Language Working Group, “Common Workflow Language (CWL) Workflow Description, v1.2.1,” 2024. [Online]. Available: <https://www.commonwl.org/v1.2/Workflow.html>
- [8] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free coordination for Internet-scale systems,” in Proc. USENIX ATC, 2010. [Online]. Available: <https://www.usenix.org/conference/usenix-atc-10/zookeeper-wait-free-coordination-internet-scale-systems>
- [9] A2A Protocol Working Group, “Agent2Agent (A2A) Protocol Specification,” version 1.0.0, 2026. [Online]. Available: <https://a2a-protocol.org/latest/specification/>
- [10] Model Context Protocol, “Model Context Protocol Specification,” revision 2026-07-28, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification/2026-07-28>
- [11] P. Bryan, K. Zyp, and M. Nottingham, “JavaScript Object Notation (JSON) Pointer,” RFC 6901, Apr. 2013, doi: 10.17487/RFC6901.
- [12] G. Klyne and C. Newman, “Date and Time on the Internet: Timestamps,” RFC 3339, Jul. 2002, doi: 10.17487/RFC3339.
- [13] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015, doi: 10.6028/NIST.FIPS.180-4.
- [14] H. Garcia-Molina and K. Salem, “Sagas,” ACM SIGMOD Record, vol. 16, no. 3, pp. 249–259, Dec. 1987, doi: 10.1145/38713.38742.